

Task AB13. VISION

5 s 1024 MB

Author: Emil Indzhev

The USS Enterprise spaceship (with Sashka as its new captain) is traveling throughout the universe, which is an infinite 2D grid of sectors. In each sector (X, Y) ¹ there is a beacon with vision strength $V_{X,Y}$. When in that sector, the ship uses the beacon to scan all sectors (X', Y') with $X - V_{X,Y} \leq X' \leq X + V_{X,Y}$ and $Y - V_{X,Y} \leq Y' \leq Y + V_{X,Y}$. Thus, the seen sectors form a square of side length $2V_{X,Y} + 1$. For each such sector, the only information seen is the vision strength of the beacon in it, i.e. $V_{X',Y'}$ (this is because beacons act as both transmitters and receivers). After scanning, the ship then teleports to one of these seen sectors, as beacons also act as teleporters.

Unfortunately, the beacons have one critical flaw: they may flip the X axis of the scan, or the Y axis, or both (i.e. there are 4 possible scan orientations). Notice that the ship uses this possibly flipped scan to decide which sector to teleport to, but the teleportation also uses the same orientation, e.g. if the Y axis is flipped and the ship teleports to the left direction (decreasing Y) of the scan, it will actually teleport to the right. This means that the ship will indeed teleport to sector chosen in the scan.

Furthermore, the ship has no memory, so its decision on where to teleport to must depend only on the scan produced by the beacon. Additionally, the ship's strategy must be deterministic - if given the same scan, it must choose to teleport to the same sector in the scan.

The ship starts at unknown coordinates and must always (regardless of where it starts and regardless of what orientations of scans it gets) be able to travel in the direction of increasing X and Y arbitrarily far (we can say that we require it to eventually reach a sector (X, Y) such that $X, Y \geq 10^{100}$). However, for this to be possible, both the travel strategy of the ship and the layout of vision strengths across the universe are crucial.

This is where you come in - you have to design an N by M rectangular pattern P of vision strengths which will repeat infinitely across the universe: $V_{X,Y} = P_{X \bmod N, Y \bmod M}$. You also have to design the travel strategy for the ship, i.e. a function which takes in the scan produced by the beacon and chooses to which of those sectors to teleport to next.

Since beacons with higher vision strength are expensive, your goal is to produce a valid solution, while trying to minimize the average vision strength in your pattern, i.e. the average value of P .

Since this problem may seem quite challenging, you decide to start with a practice problem: the same problem but in 1D. In this simpler problem, we throw away the X dimension and only use the Y one. Thus, your pattern is just a sequence of length M and it is repeated like so: $V_Y = P_{Y \bmod M}$. The scans produced by beacons are also just sequences of length $2V_Y + 1$ (containing the sectors Y' such that $Y - V_Y \leq Y' \leq Y + V_Y$). Here, there are only 2 possible orientations for the scan (regular or flipped) and the ship must be able to reach $Y \geq 10^{100}$.

Your solutions for the 1D case and the 2D case will be scored separately and you may

¹ X is row number (increasing in the down direction) and Y is the column number (increasing in the right direction).

get points for one without having a valid solution for the other.

Implementation details

You have to implement 4 functions: `getVisionPattern1d`, `getMove1d`, `getVisionPattern2d` and `getMove2d`. The first 2 are used for the 1D case and the second 2 - for the 2D one. In any test only one of these pairs of functions will be called, but all 4 must be implemented (even if the implementations for one pair are empty) for the solution to be valid (to compile).

```
std::vector<int> getVisionPattern1d()
```

This function will be called once, if the test is for the 1D case. It must return a the sequence P of length M - the 1D vision strength pattern to be repeated. M and all elements of P must be between 1 and 60.

```
int getMove1d(std::vector<int> v)
```

This function will be called once per possible scan, if the test is for the 1D case. It is given a scan produced by a beacon, as a sequence of length $2V + 1$ (where V is the vision strength of the beacon in the central sector in the sequence). It must return which sector the ship should teleport to, as an index T in the sequence, such that $0 \leq T < 2V + 1$.

```
std::vector<std::vector<int>> getVisionPattern2d()
```

This function will be called once, if the test is for the 2D case. It must return a rectangular table P of size N by M - the 2D vision strength pattern to be repeated. N , M and all elements of P must be between 1 and 60.

```
std::pair<int, int> getMove2d(std::vector<std::vector<int>> v)
```

This function will be called once per possible scan, if the test is for the 2D case. It is given a scan produced by a beacon, as a square table of size $2V + 1$ by $2V + 1$ (where V is the vision strength of the beacon in the central sector in the square). It must return which sector the ship should teleport to, as a pair of indices S and T in the table, such that $0 \leq S, T < 2V + 1$.

For the given dimensionality D (1 or 2) of the test, the grader will check that your program produces a valid pattern - that it makes valid teleportation choices for all possible scans and that your solutions works regardless of starting coordinates and sequence of orientations of scans.

Constraints

- $1 \leq D \leq 2$
- $1 \leq N, M, V_{X,Y}, V_Y \leq 60$

Subtasks and scoring

Subtask	Points	D	A^*
1	24	1	1.5
2	76	2	1.125

Your score for a given subtask (if your solution for it is correct) depends on the average vision strength in your pattern P for it. Let that be A and let the target for the subtask be A^* . Your score (fraction of points for the subtask) will be:

$$1 - \left(1 - \frac{A^* - 1}{\max(A, A^*) - 1} \right)^{0.8}$$

Sample pattern

Suppose your solution returns the following pattern with $N = 3$ and $M = 2$:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

Now, let's examine the possible scans at sector $(0, 1)$, which has vision strength 2. There are 4 possible orientations (some of which produce the same scan on this pattern):

Orientation 0: No flips

$$\begin{bmatrix} 4 & 3 & 4 & 3 & 4 \\ 6 & 5 & 6 & 5 & 6 \\ 2 & 1 & 2 & 1 & 2 \\ 4 & 3 & 4 & 3 & 4 \\ 6 & 5 & 6 & 5 & 6 \end{bmatrix}$$

Orientation 1: Y-axis flipped

$$\begin{bmatrix} 4 & 3 & 4 & 3 & 4 \\ 6 & 5 & 6 & 5 & 6 \\ 2 & 1 & 2 & 1 & 2 \\ 4 & 3 & 4 & 3 & 4 \\ 6 & 5 & 6 & 5 & 6 \end{bmatrix}$$

Orientation 2: X-axis flipped Orientation 3: Both axes flipped

$$\begin{bmatrix} 6 & 5 & 6 & 5 & 6 \\ 4 & 3 & 4 & 3 & 4 \\ 2 & 1 & 2 & 1 & 2 \\ 6 & 5 & 6 & 5 & 6 \\ 4 & 3 & 4 & 3 & 4 \end{bmatrix}$$

$$\begin{bmatrix} 6 & 5 & 6 & 5 & 6 \\ 4 & 3 & 4 & 3 & 4 \\ 2 & 1 & 2 & 1 & 2 \\ 6 & 5 & 6 & 5 & 6 \\ 4 & 3 & 4 & 3 & 4 \end{bmatrix}$$

Since 0 and 1 are identical and since the ship is deterministic, only one call to `getMove2d` will be made for them. Suppose your solution returns the move $(0, 1)$ on this scan. On orientation 0 this would mean moving 1 to the left and 2 up, while on orientation 1 - moving 1 to the right and 2 up.

Sample grader

The sample grader's first input is D . After that it will do all calls to your function: first getting the pattern P and then caching all teleportation choices. After that, it will start a console interaction, asking for the starting coordinates of the ship. After that, it will print the current state: coordinates and "raw" (without flips) scan. Then, it will ask for the orientation to be used (0 for no flips, 1 to flip the Y axis, 2 to flip the X axis and 3 to flip both). After that, it will print the scan in the given orientation, as well as what move the ship choose. The ship will be moved and this will repeat. Notice that the sample grader will not automatically verify that your solution is correct.